

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



C PROGRAMMING ASSIGNMENT

Student Name	SUDHIR MADHAVAN K
Register Number	1925260205
Course Code & Name	CSA0211
Slot	D
Assignment / Task Title	**Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative**
GitHub Repository URL	https://github.com/sudhirmadhavan/CSA0211-C-PROGRAMING.git
Submission Date	27.10.2026

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

1. Aim

To analyse the billing requirements, identify and justify the inputs, outputs, constants and intermediate values, and define assumptions and validation rules for accurate electricity billing.

2. Concept / Theory

The program compares the previous and current meter readings to calculate units consumed. It calculates the energy charge according to tariff slabs and adds a fixed charge. The solution must prevent negative consumption when a meter rolls over, avoid floating-point currency errors, and support processing from village scale to district scale.

2.1 Input Data and Justification

Input data	Purpose and justification	Suitable representation
Household number	Identifies the customer and links the calculated bill to the correct account.	unsigned integer or character array, depending on ID format
Previous reading	Stores the last cumulative meter value and is the baseline for the consumption calculation.	unsigned short if the meter is verified as 16-bit
Current reading	Stores the latest cumulative meter value for comparison with the previous reading.	unsigned short if the meter is verified as 16-bit
Household category	Allows the system to select a category-specific tariff if policy requires one.	enum
Billing period	Identifies the period and helps prevent duplicate or mismatched billing.	date/time fields or validated string
Tariff settings	Contains slab boundaries, rates and fixed charge for consistent calculation.	named integer constants or a tariff structure

2.2 Output Data and Justification

Output	Purpose and rule
Units consumed	The difference between readings, adjusted for one rollover when required.
Energy charge	The amount calculated from the tariff slabs, stored in integer paise.
Fixed charge	The fixed fee, assumed to be 5000 paise or Rs. 50.00.
Total bill	Energy charge plus fixed charge; must not be negative.
Household / billing period	Identifies the customer and period to which the bill applies.

2.3 Constants

- SLAB1_UNITS = 100: upper boundary of the free slab.
- SLAB2_UNITS = 200: upper boundary of the second slab.
- RATE1_PAISE = 0: the first 100 units are free.
- RATE2_PAISE = 225: rate for units 101–200.
- RATE3_PAISE = 450: rate for units above 200.
- FIXED_CHARGE_PAISE = 5000: fixed charge of Rs. 50.00.

These values should be defined once rather than typed repeatedly throughout the program. This ensures that a tariff revision can be applied consistently.

2.4 Negative, Fractional and Very Large Values

Valid readings and consumption should be non-negative. A raw subtraction can become negative if rollover is ignored, but valid consumption must never be negative. Currency rates may be written as decimal rupees, but integer paise provides exact representation for whole-paisa amounts. District-level totals may be very large, so a sufficiently wide integer type and overflow checks are required.

2.5 Billing Rules

1. Validate meter readings within 0–65535 for the assumed 16-bit register.
2. Calculate consumption correctly for normal readings and a single rollover.
3. Charge the first 100 units at zero energy cost.
4. Charge units 101–200 at 225 paise per unit.
5. Charge units above 200 at 450 paise per unit.
6. Add the fixed charge of 5000 paise.
7. Ensure total bill equals energy charge plus fixed charge.
8. Ensure the bill is never negative and monetary values follow a stated rounding policy.
9. Associate each bill with the correct household and billing period.

2.6 Assumptions

- The meter uses a 16-bit unsigned register with values 0 through 65535.
- At most one rollover occurs between two consecutive readings.
- The same example tariff applies to all households in the basic design.
- The fixed charge is Rs. 50.00 per household.
- Readings are validated and belong to the same household and billing period.
- Tariff values remain unchanged during one billing run.

3. Procedure / Algorithm

10. Identify the customer data and readings required for billing.
11. Separate inputs, outputs, constants and intermediate calculations.
12. Define the tariff boundaries, rates and fixed charge.
13. Specify the normal consumption and rollover calculations.
14. Define validation rules for readings, customer IDs and monetary amounts.
15. Estimate the processing and storage needs as the number of households grows.
16. Compare integer readings with the alternative string representation.

4. Program / Source Code

The following program demonstrates normal consumption and a single 16-bit meter rollover.

```
#include <stdio.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   return (unsigned int)(meter_t)(current - previous);
}

int main(void)
{
   printf("Normal consumption: %u units\n",
         consumption(1200, 1530));
   printf("Rollover consumption: %u units\n",
         consumption(65400, 120));
   return 0;
}
```

5. Output

```
Normal consumption: 330 units
Rollover consumption: 256 units
```

Normal case: 1530 − 1200 = 330 units. Rollover case: 65536 − 65400 = 136 units before rollover; 120 units after rollover. Total consumption = 136 + 120 = 256 units.

6. Alternative Concept and Complexity Analysis

If readings were stored as strings, such as "01200" and "01530", each string would be converted to an integer using strtoul() before subtraction. Let d be the number of digits in a reading. Conversion and worst-case string comparison require O(d) time.

Representation	Time per reading	Space per reading	Comparison
Fixed-width integer	O(1) arithmetic	Typically 2 bytes for 16-bit unsigned short	Compact and efficient for arithmetic.
Character string	O(d) conversion and O(d) worst-case comparison	At least d + 1 bytes including the null terminator	Preserves leading zeros but needs conversion and validation.

For n households, integer arithmetic takes O(n) total time, while converting two readings per household takes O(nd) time. Integer representation uses less space for fixed-width meter readings.

7. Analysis / Discussion

Each input and output has a clear role in producing an auditable bill. Household identifiers prevent bills from being assigned to the wrong customer; readings determine consumption; constants ensure consistent pricing; and integer paise avoids common currency precision problems. Correct rollover handling prevents negative consumption. Processing each household independently takes O(n) time for n households.

8. Result

The billing problem was analysed and the inputs, outputs, constants, rules and assumptions were justified. Normal and rollover consumption were demonstrated, and the time and space costs of integer and string representations were compared.

TASK 2 – Choose the Data Types (4 Marks)

1. Aim

To select suitable C data types for billing data, inspect their sizes and ranges using sizeof, limits.h and float.h, complete the supplied rollover code without if–else, and compare integer-paise calculations with floating-point rupees.

2. Concept / Theory

A suitable type must provide enough range and precision without wasting memory. Type sizes vary between C implementations, so the actual system must be checked with sizeof and the standard limit macros.

2.1 Data Type Selection

Data item	Suggested type	Reason
Household number	unsigned int or char array	A numeric ID can be an integer; IDs with letters or leading zeros need a string.
Previous/current reading	unsigned short	Suitable only if the meter register is confirmed as 16-bit.
Units consumed	unsigned int	Consumption is non-negative and may exceed the current reading after rollover.
Household category	enum	Provides readable named category values.
Tariff limits	int constants	Small, fixed unit boundaries.
Tariff rates	integer paise constants	Exact whole-paisa rates.
Bill per household	long long	Provides a wider range for charges.
District cumulative total	long long with overflow checks	Large aggregate amounts need a wide representation.
Fractional measurements	float or double when needed	Appropriate for measurements, but not preferred as the main currency representation.

2.2 Program to Check Sizes and Ranges

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
   printf("CHAR_BIT = %d\n", CHAR_BIT);
   printf("sizeof(char) = %zu bytes\n", sizeof(char));
   printf("sizeof(short) = %zu bytes\n", sizeof(short));
   printf("sizeof(unsigned short) = %zu bytes\n", sizeof(unsigned short));
   printf("sizeof(int) = %zu bytes\n", sizeof(int));
   printf("sizeof(unsigned int) = %zu bytes\n", sizeof(unsigned int));
   printf("sizeof(long) = %zu bytes\n", sizeof(long));
   printf("sizeof(long long) = %zu bytes\n", sizeof(long long));
   printf("sizeof(float) = %zu bytes\n", sizeof(float));
   printf("sizeof(double) = %zu bytes\n", sizeof(double));

   printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
   printf("INT_MIN = %d, INT_MAX = %d\n", INT_MIN, INT_MAX);
   printf("UINT_MAX = %u\n", UINT_MAX);
   printf("LONG_MAX = %ld\n", LONG_MAX);
   printf("LLONG_MAX = %lld\n", LLONG_MAX);
   printf("FLT_MAX = %e, FLT_EPSILON = %e\n", FLT_MAX, FLT_EPSILON);
   printf("DBL_MAX = %e, DBL_EPSILON = %e\n", DBL_MAX, DBL_EPSILON);
```

```
    return 0;
}
```

2.3 Typical Sizes and Ranges

Type	Typical size	Typical range / note
char	1 byte	Usually −128 to 127 or 0 to 255 depending on implementation.
short	2 bytes	Often −32768 to 32767.
unsigned short	2 bytes	Often 0 to 65535; verify USHRT_MAX.
int	4 bytes	Often −2147483648 to 2147483647.
unsigned int	4 bytes	Often 0 to 4294967295.
long	4 or 8 bytes	Depends on platform.
long long	8 bytes	Typically signed 64-bit range.
float	4 bytes	Approximate single-precision floating point.
double	8 bytes	Approximate double-precision floating point.

The actual output from the size/range program is the evidence to include in the report; the table lists common values rather than guarantees for every C implementation.

2.4 Integer Paise Versus Float Rupees

Many decimal fractions cannot be represented exactly in binary floating point. Repeatedly adding Rs. 0.10 as float may therefore produce a total slightly different from Rs. 400.00. Instead, store Rs. 0.10 as 10 paise and add the integer 10 exactly 4,000 times. Integer paise is exact for whole-paisa values as long as the integer type does not overflow.

3. Procedure / Algorithm

17. Include stdio.h and limits.h.
18. Define meter_t as unsigned short.
19. Return the subtraction result converted back to meter_t, without if–else.
20. Print sizeof(meter_t) and USHRT_MAX.
21. Test normal readings and rollover readings.
22. Run the type-size program and record actual output.
23. Compare float and integer-paise accumulation for 4,000 bills.

4. Complete the Code – TODO 1

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned int)(meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
```

```
    return 0;
}
```

5. Expected Output and Explanation

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

A 16-bit unsigned register has 65536 possible values. Conversion back to the unsigned meter type gives the subtraction result modulo 65536. For 65400 to 120, the result is 256, not a negative number. This relies on unsigned short being 16 bits and assumes no more than one rollover between readings.

6. Float Precision Test

```
#include <stdio.h>

int main(void)
{
    float total_float = 0.0f;
    long long total_paise = 0;

    for (int i = 0; i < 4000; i++) {
        total_float += 0.10f;
        total_paise += 10;
    }

    printf("Float total: %.9f\n", total_float);
    printf("Integer total: Rs %lld.%02lld\n",
           total_paise / 100, total_paise % 100);
    return 0;
}
```

A common IEEE 754 single-precision GCC build may produce a float total near 400.015197754, whereas the integer calculation gives Rs. 400.00 exactly. The exact floating-point output can vary by compiler and settings.

7. Alternative Concept and Complexity Analysis

If money were stored as double rupees, the program could add floating-point charges and round the final total to the nearest paisa with an explicit rule, such as round(total * 100.0) / 100.0. Printing with %.2f alone changes the displayed value but does not round the stored value.

Feature	Integer paise	Double rupees
Time per addition	O(1)	O(1)
Time for n bills	O(n)	O(n)
Space per value	Typically 4 or 8 bytes depending on chosen integer type	Typically 8 bytes
Precision	Exact for whole-paisa amounts when no overflow occurs	Approximate binary representation
Rounding error	No representation error for whole paise	May accumulate; depends on values and order

The error does not necessarily grow linearly with n. It depends on the values, addition order, and floating-point implementation. Integer paise avoids binary representation error for whole-paisa amounts.

8. Analysis / Discussion

The chosen types match the expected data ranges. The size/range program confirms the actual system values. Rollover arithmetic works for the specified 16-bit counter, while integer paise prevents common floating-point currency errors. Wide integer totals should still be checked for overflow at district scale.

9. Result

Suitable data types were selected and justified. The supplied consumption function was completed without if–else and produced 330 units for normal readings and 256 units for rollover readings. The precision comparison demonstrated why integer paise is preferred.

TASK 3 – Define the Constants (3 Marks)

1. Aim

To define tariff limits, rates and fixed charges as named constants, complete the billing function using the supplied macros, and verify that a tariff revision is applied consistently.

2. Concept / Theory

Constants centralise fixed tariff values so the same rate is not typed in many places. #define performs preprocessor text substitution; const declares a typed read-only object; enum defines named integer constants, often used for categories or states.

2.1 Constant Naming

- Use uppercase letters and underscores for macro names, for example RATE2_PAISE.
- Include units in names where useful, such as FIXED_CHARGE_PAISE.
- Use meaningful names for slab boundaries and rates.
- Keep one authoritative definition of each tariff value.

Feature	#define	const variable	enum
Purpose	Preprocessor replacement text	Typed read-only object	Named integer constants
Type checking	No type for the macro itself	Normal type checking	Integer-based
Scope	Depends on preprocessing and source placement	Normal C scope rules	Normal C scope rules
Debugger visibility	Not a run-time object	Often inspectable	Enumerator names may be available
Storage	Macro itself uses no object storage	May use storage if emitted	Enumerator itself uses no object storage
Best fit	Required by the supplied TODO program	Typed configuration objects	Categories and states

3. Procedure / Algorithm

24. Define the supplied slab limits, rates and fixed charge.
25. Initialise the bill with the fixed charge in paise.
26. If units exceed 200, charge units above 200 at RATE3_PAISE and the complete second slab at RATE2_PAISE.
27. Otherwise, if units exceed 100, charge only units above 100 at RATE2_PAISE.
28. Return the bill in paise.
29. Print the amount in rupees and two-digit paise.
30. Change RATE2_PAISE from 225 to 230, recompile and verify all test outputs.

4. Complete the Code – TODO 1

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
```

```
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        paise += (long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);

    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

5. Expected Output

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

6. Tariff Revision Test

Change the definition to #define RATE2_PAISE 230 and recompile the program. The expected output is:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

For 350 units, the second slab contains 100 units. Increasing its rate by 5 paise per unit adds 500 paise, or Rs. 5.00, to the total.

7. Alternative Concept and Complexity Analysis

If the rates were const variables, they would have types and may be visible in a debugger. A const object does not replace a macro in every context requiring a compile-time integer constant expression in C. enum is suitable for household categories but is not the clearest representation for monetary rates.

The billing function performs a fixed number of operations, so it takes O(1) time and O(1) auxiliary space per bill. Processing n households takes O(n) time. Changing from #define to const does not change the algorithmic running time. A macro itself occupies no object storage; a const object may occupy storage depending on use and optimisation.

8. Analysis / Discussion

The tariff values are defined once and the program calculates all charges in integer paise. The test values verify the free first slab, the second slab, and the third slab. Changing RATE2_PAISE in one place applies the revised rate consistently after recompilation.

9. Result

The supplied compute_bill() function was completed using the defined tariff macros. The original and revised tariff outputs were compared with the expected values.

TASK 4 – Use Macros Safely (4 Marks)

1. Aim

To implement safe macros for repeated calculations, inspect macro expansion using gcc -E, explain operator-precedence and repeated-evaluation problems, and compare macros with static inline functions.

2. Concept / Theory

The C preprocessor expands macro names before compilation. Parentheses around each parameter and the whole expression help preserve operator precedence. However, parentheses do not stop a macro from evaluating an argument more than once.

2.1 SQUARE Macro

Unsafe macro: #define SQUARE(x) x * x. For SQUARE(a + 1), expansion gives a + 1 * a + 1. If a = 3, the result is 3 + 3 + 1 = 7, not 16.

Safe macro: #define SQUARE(x) ((x) * (x)). It expands SQUARE(a + 1) to ((a + 1) * (a + 1)), giving 16 when a = 3.

2.2 MAX Macro

The macro #define MAX(a, b) (((a) > (b)) ? (a) : (b)) can evaluate an argument twice: once in the comparison and again in the selected expression. Therefore MAX(i++, j) can increment i more than once. Avoid side-effecting expressions as macro arguments.

3. Procedure / Algorithm

31. Define RUPEES_TO_PAISE, SQUARE and MAX macros with full parentheses.
32. Implement equivalent static inline functions.
33. Run the program with known values and check its output.
34. Run gcc -E -P main.c -o preprocessed.c.
35. Inspect the expanded macro expressions.
36. Compare the time cost and code-size implications of macros and inline functions.

4. Program / Source Code

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((r) * 100L)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

static inline int square_function(int x)
{
   return x * x;
}

static inline int max_function(int a, int b)
{
   return (a > b) ? a : b;
}

int main(void)
{
   int a = 3;
   int b = 5;

   printf("Square of a + 1: %d\n", SQUARE(a + 1));
   printf("Maximum of a and b: %d\n", MAX(a, b));
   printf("100 rupees in paise: %ld\n", RUPEES_TO_PAISE(100));
   printf("Square using inline function: %d\n",
         square_function(a + 1));
```

```
    printf("Maximum using inline function: %d\n",
           max_function(a, b));
    return 0;
}
```

5. Expected Output

```
Square of a + 1: 16
Maximum of a and b: 5
100 rupees in paise: 10000
Square using inline function: 16
Maximum using inline function: 5
```

6. Macro Expansion Using GCC

37. Save the program as main.c.
38. Run gcc -E -P main.c -o preprocessed.c.
39. Open preprocessed.c and search for the expanded expressions.
40. Compile using gcc -std=c11 -Wall -Wextra main.c -o main.
41. Run the executable and capture the output screenshot.

Expected expansion examples:

```
SQUARE(a + 1)          -> ((a + 1) * (a + 1))
MAX(a, b)              -> (((a) > (b)) ? (a) : (b))
RUPEES_TO_PAISE(100)   -> ((100) * 100L)
```

7. Alternative Concept and Complexity Analysis

Replace macros with static inline functions when normal type checking, single argument evaluation, and easier debugging are more important. The inline keyword is a request to the compiler; it does not guarantee that the function call will be removed.

```
static inline int square_function(int x)
{
    return x * x;
}

static inline int max_function(int a, int b)
{
    return (a > b) ? a : b;
}
```

Feature	Macro	static inline function
Processing	Preprocessor substitution	Compiler processes a function; may inline it
Type checking	No normal parameter type checking	Parameter types are checked
Argument evaluation	Can evaluate an argument multiple times	Each argument is evaluated once at the call site
Time per operation	O(1) for simple arithmetic	O(1); a non-inlined call may add constant overhead
Code size for k uses	Replacement text may be copied at each use	May share one function body or duplicate instructions if inlined
Debugging	Can be harder to trace	Usually easier

For k uses of a constant-time operation, total time is O(k) for either approach. Macro expansion may increase generated code size as k grows; actual size depends on compiler optimisation. Both approaches use O(1) auxiliary data space per operation, excluding machine-code storage.

8. Analysis / Discussion

Fully parenthesised macros avoid precedence errors, but MAX can still evaluate arguments more than once. The preprocessor output demonstrates how macro calls are replaced before compilation. Inline functions provide stronger type checking and safer argument evaluation, though actual inlining and code size depend on compiler decisions.

9. Result

Safe macros and equivalent inline functions were implemented. The program produced the expected square, maximum and rupees-to-paise results. Macro expansion and the time and code-size trade-offs were analysed.

DESIGN CHALLENGE – SMART METERS AND TIME-OF-DAY TARIFF

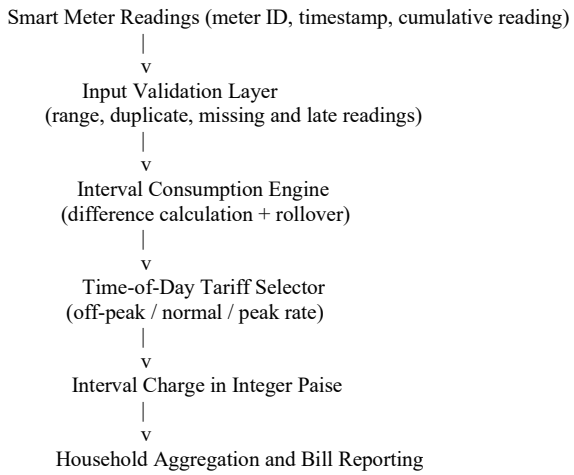
1. Aim

To decide whether the existing billing design should be retained, modified or redesigned for smart meters that report every 15 minutes and use a time-of-day tariff.

2. Decision: Modify the Original Design

The original design should be modified rather than completely replaced. Integer-paise billing, named tariff constants, validation, and rollover-aware arithmetic remain useful. The data model and billing workflow must be extended to include timestamps, interval consumption, time-of-day rate selection, and scalable storage.

3. Proposed Architecture



4. Required Changes

- Store meter ID, timestamp and cumulative reading for each report.
- Calculate interval usage from consecutive readings and handle rollover.
- Select the applicable rate using the timestamp and tariff schedule.
- Calculate each interval charge in integer paise and aggregate it for the billing period.
- Detect missing, duplicate, delayed and out-of-order readings.
- Define clearly how the existing slab tariff combines with time-of-day rates before implementation.
- Use scalable storage and processing for district-level volume.

5. Scale Estimate

A 15-minute reporting interval creates 96 readings per meter per day. Over 30 days, one household generates 2,880 intervals. For 800,000 households, this is 2,304,000,000 interval records per 30-day period if every interval is stored individually. The system therefore needs efficient ingestion, partitioned storage and aggregation.

6. Result

The original design should be modified. Its reliable integer-paise arithmetic, centralised constants and rollover handling should be retained, while timestamp-aware interval processing, time-of-day tariff selection, smart-meter validation and scalable storage should be added.

SUBMISSION CHECKLIST

- Create the repository named CSA02-<RegisterNumber>-Assignment1.
- Keep one folder per task containing source code and genuine output screenshots.
- Include README.md with your name, register number, GCC version, exact compile commands, run steps and result notes.
- Make at least one progressive commit for each task.
- Add the working GitHub link to the report cover and submission form.
- Verify the student details and replace sample identity information if necessary.

README.md – Suggested Content

```
# CSA02-<RegisterNumber>-Assignment1

Student Name: SUDHIR MADHAVAN K
Register Number: 1925260205
Course: CSA0211 – C Programming
Compiler: GCC (record the exact version from `gcc --version`)

## Task 1
gcc -std=c11 -Wall -Wextra task1.c -o task1

## Task 2
gcc -std=c11 -Wall -Wextra datatype_sizes.c -o datatype_sizes
gcc -std=c11 -Wall -Wextra float_comparison.c -o float_comparison

## Task 3
gcc -std=c11 -Wall -Wextra main.c -o task3

## Task 4
gcc -std=c11 -Wall -Wextra main.c -o task4
gcc -E -P main.c -o preprocessed.c

Add run instructions, a result note and your output screenshot for every task.
```

Before submitting, execute all programs and insert your own screenshots. Confirm the cover-page details and ensure the GitHub repository link is accessible to the faculty.